

TN-LCO --- Tamil Nadu District Land-Change Observatory

A district-scale research prototype for detecting, filtering, prioritising, and verifying land-cover change from satellite imagery.

TN-LCO is a deliberately smaller and more defensible implementation of the original **"One Tamil Nadu Geo-AI Platform"** proposal.

The original proposal describes a state-wide platform combining satellite imagery, UAVs, IoT sensors, AI/deep learning, cadastral data, a Digital Twin, WebGIS, mobile applications, predictive modelling, and near-real-time monitoring.

This repository does **not** attempt to implement all of that.

Instead, the prototype focuses on one technically meaningful problem:

Given satellite observations of the same area at two comparable periods, can we identify meaningful land-cover change while reducing false alarms caused by seasonal agricultural variation?

The prototype demonstrates that complete workflow on a manageable **Perundurai--Erode study window**.

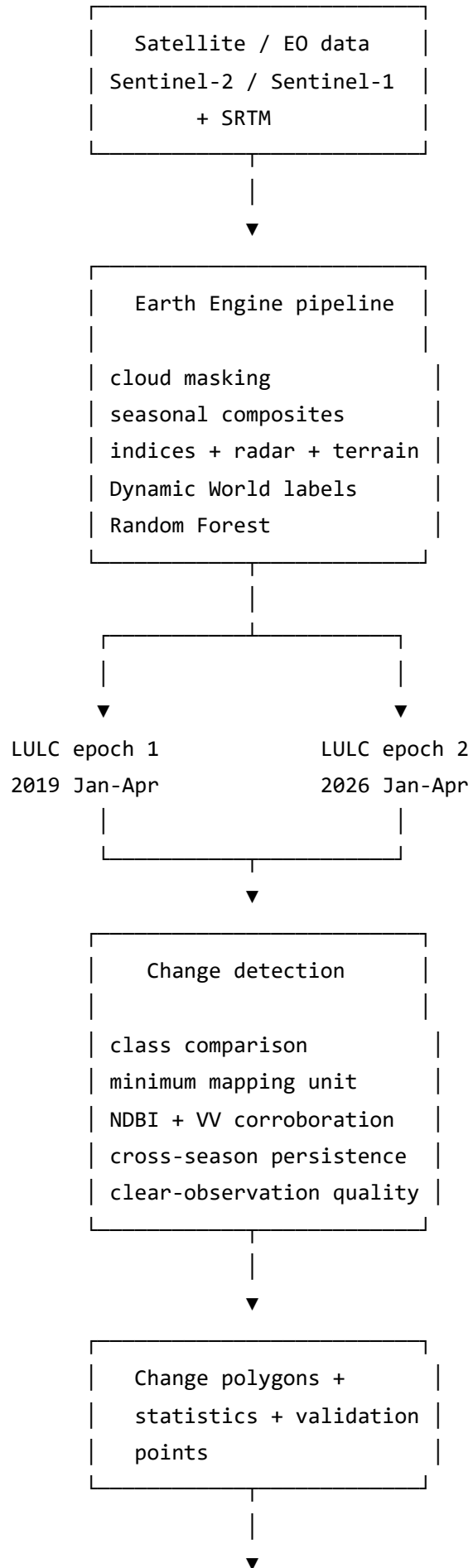
1. What this project actually does

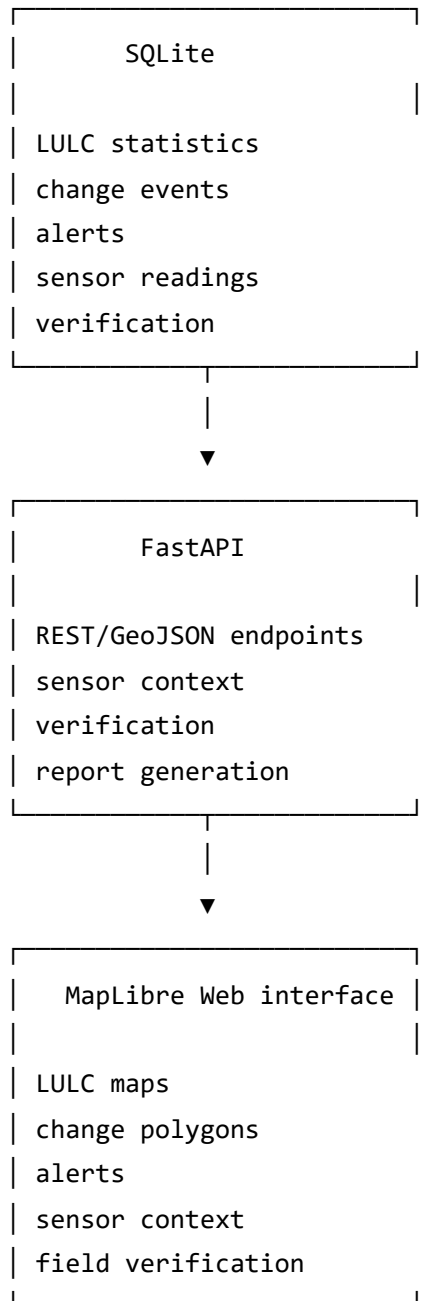
At a high level:

1. Google Earth Engine obtains free satellite observations.
2. Sentinel-2 optical imagery is cloud-masked and composited.
3. Sentinel-1 radar imagery is added because radar provides information that optical imagery cannot.
4. SRTM elevation and slope are added as terrain features.
5. Spectral indices such as NDVI, MNDWI, NDBI, NDMI, and BSI are calculated.

6. Google Dynamic World provides weak training labels.
7. A Random Forest classifier produces six land-cover classes for each epoch.
8. The two classified maps are compared.
9. Several filters remove changes that are too small, lack supporting evidence, do not persist across another season, or have poor image quality.
10. Remaining pixels are converted into change polygons.
11. The polygons and statistics are exported as JSON/GeoJSON.
12. A local Python loader stores the results in SQLite.
13. Simulated sensor readings provide environmental context around the second satellite observation period.
14. FastAPI exposes the results as JSON/GeoJSON endpoints.
15. A single-file MapLibre frontend displays the maps, changes, alerts, sensors, and verification workflow.
16. A report endpoint turns the database results into a simple district report.

The system is therefore a pipeline:





2. Why the prototype is smaller than the original proposal

The original proposal combines several independent research and engineering problems:

- state-wide land-use monitoring

- deep-learning semantic segmentation
- change detection
- time-series prediction
- IoT
- UAV mapping
- cadastral integration
- Digital Twin simulation
- WebGIS
- mobile applications
- citizen-generated data
- cloud infrastructure
- departmental integration

Trying to implement all of these in a zero-budget prototype would create a system that is broad but difficult to validate.

TN-LCO therefore concentrates on the part that can be demonstrated end-to-end:

land-cover classification → change detection → seasonal false-positive filtering → spatial alerts → verification.

This is an architectural decision, not merely a reduction in features.

A system with fewer components that actually exchange real data is more useful for research than a system containing many disconnected demos.

3. The most important design decision: land-cover change, not legality

This system detects **land-cover change**.

It does not determine:

- whether a land conversion is legal
- whether an owner has permission
- whether a building violates planning rules

- whether a parcel has been encroached upon
- who owns a piece of land
- whether zoning rules were violated

The prototype does not contain:

- cadastral parcel geometry
- ownership records
- approval records
- zoning records

Therefore a result such as:

Cropland → Built-up

means:

The classifier detected a spatial region whose mapped land-cover class changed from Cropland to Built-up between the two analysed periods.

It does **not** mean:

Illegal construction occurred.

This distinction is fundamental to the architecture.

4. Study area and epochs

The current configuration uses:

Study area:

Perundurai-Erode window

Approximate bounding box:

77.45, 11.15, 77.80, 11.45

Approximate size:

38 × 33 km

Epoch 1:

2019-01-01 → 2019-04-30

Epoch 2:

2026-01-01 → 2026-04-30

Persistence-check season:

2025-07-01 → 2025-10-31

Why Jan--Apr?

The two main epochs use the same seasonal window.

That matters because comparing:

January 2019

against:

July 2026

would mix development with seasonal vegetation differences.

A field may naturally be:

green → harvested → bare → green

without any land-use conversion.

Using comparable periods reduces that problem.

The additional 2025 July--October period is used as a **persistence check** rather than as the main comparison epoch.

5. Land-cover classes

The prototype uses six classes:

ID	Class	Meaning
0	Water	Rivers, reservoirs, ponds and other mapped water
1	Forest/Trees	Tree-dominated vegetation
2	Cropland	Agricultural/cultivated land
3	Built-up	Buildings, urban surfaces and developed surfaces
4	Barren	Bare soil and similar exposed surfaces
5	Shrub/Grass	Grassland, shrubs and related vegetation

The IDs are intentionally fixed.

A fixed numeric encoding makes the raster and database representation compact and allows transition codes to be generated.

For example:

```
from = 2
to   = 3
```

becomes:

```
23
```

The loader decodes this as:

```
from_class = transition // 10
to_class   = transition % 10
```

Therefore:

23 = Cropland → Built-up

52 = Shrub/Grass → Cropland

34 = Built-up → Barren

The transition encoding assumes class IDs remain single digits.

6. Earth observation data

6.1 Sentinel-2

Dataset:

COPERNICUS/S2_SR_HARMONIZED

Sentinel-2 is the main optical source.

Important bands:

B2 Blue

B3 Green

B4 Red

B8 Near infrared

B11 SWIR

B12 SWIR

The prototype uses surface-reflectance imagery rather than raw radiance.

Why Sentinel-2?

Advantages:

- free
- 10 m native resolution for important bands
- frequent observations

- useful visible, NIR and SWIR bands
- strong ecosystem for Earth observation processing

Alternatives include:

- Landsat 8/9
- Resourcesat
- Cartosat
- commercial high-resolution imagery

Why not use commercial imagery?

Cost.

Why not use Landsat as the primary source?

Its 30 m resolution makes the minimum mapping unit much coarser.

Why not depend on Cartosat?

Access and licensing are more complicated than a zero-budget prototype should require.

Landsat remains useful for historical extension in a future version.

7. Sentinel-2 cloud masking

The pipeline uses the Sentinel-2 Scene Classification Layer.

It removes:

- SCL 3 → cloud shadow
- SCL 8 → medium-probability cloud
- SCL 9 → high-probability cloud
- SCL 10 → cirrus
- SCL 11 → snow

The remaining image is divided by 10000 to convert the Sentinel-2 surface-reflectance integer representation to reflectance-like values.

Why mask clouds?

A cloud can look extremely different from the actual ground.

If:

```
2019 = clear field
2026 = cloud
```

a naive change detector could interpret the cloud as land-cover change.

Cloud masking is therefore a prerequisite, not an optional improvement.

8. Seasonal median compositing

For each epoch, multiple valid Sentinel-2 observations are combined using a pixel-wise median.

Conceptually:

```
Image 1 ┌┐
Image 2 ┌┐
Image 3 ┌┐
Image 4 ┌┐ → median per pixel → seasonal composite
Image 5 ┌┐
```

Why median?

Median is robust against:

- remaining outliers
- occasional cloud contamination
- abnormal observations
- individual noisy acquisitions

Alternatives:

Single image

Simpler, but fragile.

One cloudy or unusual image can dominate the result.

Mean

Sensitive to outliers.

Maximum-value composite

Useful for some vegetation applications but can exaggerate unusual observations.

Median

Good general-purpose compromise for this prototype.

9. Clear-observation count

The pipeline also calculates:

```
clear_obs
```

This counts how many valid B8 observations contributed at each pixel.

The final quality requirement is:

```
T1 clear observations >= 5  
AND  
T2 clear observations >= 5
```

Why five?

A composite based on one or two usable observations is much less trustworthy than one based on several observations.

Five is not a universal scientific constant. It is a conservative prototype threshold.

Alternatives:

- 3 observations: more coverage, weaker confidence
- 5 observations: current choice
- 8--10 observations: stronger temporal support but may remove too many pixels
- adaptive threshold: better future design

The important architectural idea is that **data quality is explicitly represented and used in change detection** rather than silently assumed.

10. Sentinel-1 radar

Dataset:

COPERNICUS/S1_GRD

The pipeline uses:

VV
VH

and filters to:

```
instrumentMode = IW
orbitProperties_pass = DESCENDING
VV available
VH available
```

A median composite is generated and a focal median filter is applied.

Why radar?

Optical imagery measures reflected electromagnetic energy.

Radar behaves differently.

Sentinel-1 can provide information related to:

- surface roughness
- structure
- moisture
- built surfaces

Most importantly, radar is not blocked by ordinary cloud in the same way as optical imagery.

This makes it useful for supporting change detection.

Why fix the orbit direction?

Mixing ascending and descending acquisitions can introduce geometry and observation-condition differences.

Using one pass direction keeps the prototype more consistent.

Alternatives:

- use both orbit directions and harmonise them
- use only VV
- use VV + VH
- use temporal statistics rather than only median

The current design chooses consistency and simplicity.

11. Terrain data

Dataset:

Features:

elevation

slope

Why include terrain?

Land-cover classes are not independent of geography.

For example:

- elevation can distinguish landscape zones
- slope can help distinguish certain terrain types
- vegetation patterns often correlate with terrain

Terrain is therefore additional contextual information.

It is not expected to be the dominant signal.

12. Spectral indices

The prototype computes five indices.

NDVI

$$\text{NDVI} = (\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red})$$

Used primarily for vegetation.

MNDWI

$$\text{MNDWI} = (\text{Green} - \text{SWIR}) / (\text{Green} + \text{SWIR})$$

Useful for water detection.

NDBI

$$\text{NDBI} = (\text{SWIR} - \text{NIR}) / (\text{SWIR} + \text{NIR})$$

Useful as a built-up indicator.

NDMI

$$\text{NDMI} = (\text{NIR} - \text{SWIR}) / (\text{NIR} + \text{SWIR})$$

Useful for vegetation/land moisture context.

BSI

The prototype uses:

$$\begin{aligned} \text{BSI} = & \\ & (\text{B11} + \text{B4} - \text{B8} - \text{B2}) \\ & / \\ & (\text{B11} + \text{B4} + \text{B8} + \text{B2}) \end{aligned}$$

Useful for exposed/bare surfaces.

Why indices are used in addition to raw bands

Raw bands contain information, but indices provide engineered relationships that often make specific land characteristics easier for a classifier to separate.

Instead of asking the Random Forest to discover every useful relationship itself, we provide physically meaningful derived features.

Alternative:

Train directly on raw bands.

That is possible, but the current approach is easier to interpret and requires less model complexity.

13. The feature stack

The classifier receives:

B2

B3

B4

B8

B11

B12

NDVI

MNDWI

NDBI

NDMI

BSI

VV

VH

elevation

slope

That is 15 predictors.

The important design principle is **feature diversity**:

Optical reflectance
+
Spectral indices
+
Radar
+
Terrain

This is preferable to simply adding more neural-network layers.

14. Why Random Forest instead of a deep-learning model?

The original proposal lists:

- U-Net
- DeepLabV3+
- Vision Transformer
- Swin Transformer
- SAM
- LSTM

The prototype intentionally does not use these as the primary classifier.

It uses:

Random Forest

with:

100 trees

Why?

Random Forest is:

- fast
- relatively easy to train
- robust to mixed features
- suitable for tabular pixel features
- available directly inside Earth Engine
- easier to reproduce
- easier to debug
- a strong baseline

The research question is not:

Can we use the newest neural network?

It is:

Can we reliably detect useful land-cover change and reduce seasonal false alarms?

A complex model is only justified if it produces a measurable improvement.

15. Why not U-Net in the current prototype?

U-Net is a good future option because it can use neighbourhood/spatial context.

However, it introduces:

- training-image preparation
- patch extraction
- GPU requirements
- augmentation

- model checkpointing
- inference infrastructure
- additional validation complexity

For the current prototype, that engineering cost is not necessary.

A future experiment can compare:

Random Forest

vs

U-Net

and report whether the additional complexity actually improves independent accuracy.

16. Why not LSTM?

LSTM is intended for temporal sequence modelling.

The original proposal suggests LSTM for future land-use prediction.

The problem is data volume.

A few annual land-cover maps do not form a sufficiently long sequence for a convincing LSTM study.

An LSTM trained on a handful of observations can easily become a sophisticated-looking model with weak statistical support.

A future prediction system should instead start with:

- transition matrices
- suitability modelling
- Random Forest drivers
- cellular allocation

and only introduce deep temporal models when enough historical observations exist.

17. Training labels: Dynamic World

The prototype uses:

GOOGLE/DYNAMICWORLD/V1

as weak labels.

This is an important limitation.

Dynamic World is itself a model-derived land-cover product.

Therefore:

```
Dynamic World label
    ↓
Random Forest training
    ↓
Random Forest classification
```

does not constitute independent ground truth.

If the Random Forest agrees with Dynamic World, that demonstrates:

| agreement with the reference product.

It does not demonstrate:

| true accuracy on the ground.

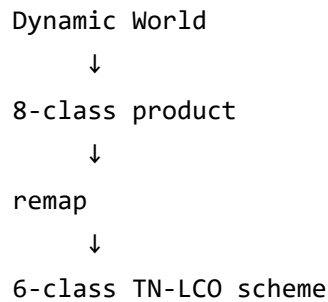
The pipeline therefore exports independent validation points for human interpretation.

18. Dynamic World class mapping

Dynamic World has more classes than this prototype.

The pipeline maps them into the six project classes.

Conceptually:



This is necessary because the classifier and change detector operate on the project's six-class taxonomy.

The remapping is a modelling decision.

For example, flooded vegetation is mapped into the project's agricultural class because the prototype treats that type of seasonal agricultural vegetation as cropland.

This should be revisited if a future version focuses specifically on wetlands.

19. Training parameters

The pipeline uses:

```
SAMPLES_PER_CLASS = 400  
RF_TREES = 100
```

400 samples per class

This attempts to give each class comparable representation.

Why stratified sampling?

Without stratification, common classes such as vegetation or barren surfaces could dominate the training sample.

Why 400?

It gives a manageable training set while avoiding a huge server-side sample.

Alternatives:

- 100/class: faster, less stable
- 400/class: current balance
- 1000/class: more data but greater processing
- proportional sampling: better for natural prevalence but can worsen class imbalance

20. Train/test split

The current classifier creates a random column:

```
r < 0.7 → training  
r >= 0.7 → testing
```

Therefore approximately:

```
70% training  
30% testing
```

This is useful as an initial model check.

However, random pixel splitting can suffer from **spatial autocorrelation**.

Nearby pixels are not independent observations.

A future rigorous study should therefore add spatially separated validation blocks.

The current reported confusion matrix should be treated as model agreement diagnostics, not final field accuracy.

21. The actual change-detection method

The prototype uses:

```
post-classification comparison
```

rather than an end-to-end deep-learning change detector.

The first operation is simply:

```
changed_raw = cls_t1 != cls_t2
```

This means:

```
2019 class != 2026 class
```

is a candidate change.

Everything after this is filtering.

22. Why post-classification comparison?

An alternative is to train a dedicated change-detection neural network.

Examples include:

- Siamese networks
- ChangeStar-like architectures
- transformer-based change detectors
- image-difference CNNs

Those can be powerful but require carefully labelled change datasets.

Post-classification comparison is:

- explainable
- easy to debug
- easy to visualise
- compatible with weak LULC labels
- easy to describe to planners

Most importantly, it lets the system say exactly what changed:

Cropland → Built-up

rather than only:

change = true

23. Change filter A: minimum mapping unit

The prototype uses:

`MIN_MAPPING_UNIT_PIXELS = 10`

At Sentinel-2's 10 m resolution:

$$10 \text{ m} \times 10 \text{ m} = 100 \text{ m}^2$$

Therefore:

$$\begin{aligned} 10 \text{ pixels} \times 100 \text{ m}^2 &= 1,000 \text{ m}^2 \\ &= 0.1 \text{ ha} \end{aligned}$$

The filter removes changes smaller than approximately:

0.1 hectare

Why?

Classification noise often appears as isolated pixels or tiny fragments.

Without a minimum mapping unit, the map can become:

pixel
pixel
pixel
pixel
tiny polygon
tiny polygon

instead of meaningful objects.

Why not use 1 hectare?

That would be too aggressive for this study.

Why not use 0.01 hectare?

Sentinel-2 cannot reliably resolve such small objects.

The 0.1 ha threshold is therefore a practical prototype compromise.

This does **not** mean every 0.1 ha object is reliably classified.

24. Change filter B: built-up spectral corroboration

For areas classified as Built-up in epoch 2, the prototype requires:

NDBI delta > 0.05

AND

VV delta > 0.5 dB

Conceptually:

new built-up classification

+

higher NDBI

+

higher radar backscatter

↓

stronger evidence for built-up conversion

Why two signals?

A classifier can make mistakes.

If only the class label changes, the system has one piece of evidence.

If the class label changes and independent optical/radar signals also change in the expected direction, confidence increases.

Why 0.05 NDBI?

It is a simple, interpretable prototype threshold.

It is not a universal physical constant.

Why 0.5 VV?

Same principle.

It provides a radar corroboration threshold.

These values should be calibrated using independent validation in a research version.

25. Change filter C: cross-season persistence

This is the most important methodological idea in TN-LCO.

The prototype computes a separate classification for:

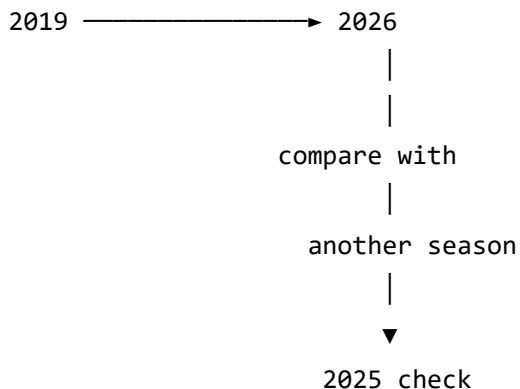
2025 July–October

and requires:

`class_2025_check == class_2026`

for a retained change.

The idea is:



Suppose a field is:

2019: Cropland
2026 Jan-Apr: Barren
2025 Jul-Oct: Cropland

That can be consistent with a seasonal crop-cycle effect.

But if:

2019: Cropland
2026 Jan-Apr: Built-up
2025 Jul-Oct: Built-up

the change is more persistent.

Why this matters in Tamil Nadu

Agricultural land can change appearance dramatically between seasons.

A simple two-date image comparison can therefore produce many false alarms.

The persistence check attempts to distinguish:

temporary appearance change

from:

persistent land-cover change

This is the central reason the project uses a second season.

26. Change filter D: data quality

The final mask requires:

T1 clear observations ≥ 5
T2 clear observations ≥ 5

This prevents low-observation pixels from producing change events.

The full final condition is therefore approximately:

raw change
AND
large enough
AND
spectral evidence is sufficient
AND
persists in another season
AND
both epochs have adequate observations

Only then does the system retain the change.

27. Raw change vs retained change

The pipeline records both:

change_pixels_raw
change_pixels_kept

This is important.

If a system only reports:

1.8 million changed pixels

the user cannot know how aggressive the filtering was.

Reporting both gives:

raw candidate changes
↓
filtering
↓
retained changes

The filter ratio becomes a measurable part of the method.

For example, the current successful run produced approximately:

Raw changed pixels:	5,844,422
Retained pixels:	1,853,066

That is roughly 31.7% retained and 68.3% removed.

These values belong to the current dataset/configuration and should not be treated as universal performance numbers.

28. Vectorisation

The final change raster is converted into polygons using:

```
reduceToVectors()
```

Each connected region becomes a vector feature.

Each feature contains:

```
transition  
ndbi_d  
vv_d  
area_ha  
geometry
```

Why vectorise?

Raster pixels are excellent for computation.

Humans and APIs often need objects.

A polygon can be described as:

Cropland → Built-up

Area: 2.43 ha

NDBI change: 0.11

VV change: 1.2

That is much easier to rank, display, verify, and report than thousands of individual pixels.

29. Why polygons are limited

The pipeline limits the exported change polygons.

This protects the Earth Engine export from becoming excessively large.

The prototype previously encountered server-side export/serialization limits, so the vector export was intentionally made lightweight.

This is an engineering trade-off:

more polygons
↓
more detail
↓
larger export
↓
higher failure risk

versus:

fewer largest polygons
↓
smaller export
↓
more reliable prototype

For a production system, this should be replaced by tiled/vector-tile processing rather than simply raising the limit.

30. Why Earth Engine is used

The heavy geospatial processing runs in:

Google Earth Engine

instead of locally.

This includes:

- satellite catalogue access
- image filtering
- compositing
- index calculation
- radar processing
- terrain extraction
- sampling
- Random Forest training
- classification
- change masks
- vectorisation
- statistics

Why?

The alternative is to download and process large amounts of satellite imagery locally.

That introduces:

- storage requirements
- download time
- memory requirements
- preprocessing complexity
- CPU/GPU requirements

Earth Engine allows the prototype to keep most of that work server-side.

31. Why the local application is deliberately simple

The original proposal suggests:

- cloud platform
- scalable database
- WebGIS
- mobile application
- state-wide service

The prototype uses:

```
Python
+
SQLite
+
FastAPI
+
single HTML/JavaScript frontend
```

No:

- Docker
- PostgreSQL server
- PostGIS
- Redis
- Kubernetes
- separate frontend build system
- cloud VM
- paid GIS software

This is intentional.

The prototype's purpose is to demonstrate the research workflow, not infrastructure engineering.

32. Why SQLite?

The database is:

SQLite

instead of PostgreSQL/PostGIS.

Advantages for this prototype

SQLite:

- ships with Python
- requires no server
- requires no password
- requires no database installation
- works offline
- is easy to reset
- is easy to inspect
- is sufficient for hundreds or a few thousand change events

That makes it ideal for a zero-budget local prototype.

Why not PostGIS?

PostGIS would be the correct production architecture.

It provides:

- spatial indexes
- spatial joins
- geometry validation
- projected area calculations
- proximity queries
- vector tiles
- large-scale concurrent access

But installing and operating PostgreSQL/PostGIS introduces infrastructure that does not help prove the central research idea.

Therefore:

Prototype → SQLite

Production → PostgreSQL + PostGIS

33. Why geometry is stored as GeoJSON text

SQLite has no native PostGIS geometry type.

Therefore change geometry is stored as JSON text.

The loader also stores:

```
centroid_lon  
centroid_lat
```

at load time.

This is a deliberate compromise.

The API can then:

- return polygons as GeoJSON
- calculate nearest sensor distance using the centroid
- avoid requiring a spatial database

The downside is that SQLite cannot perform proper spatial queries.

For example:

```
find all changes within 500 m of a water body
```

is much better implemented in PostGIS.

34. Sensor architecture

The prototype contains four simulated sensor nodes and seeded historical readings.

The readings are explicitly:

SIMULATED

They are not real hardware observations.

The UI must retain this marking.

The sensor layer contains values such as:

- rainfall
- soil moisture
- temperature
- humidity

Why simulate sensors?

The project proposal requires an AIoT component, but the prototype has no deployed physical sensor network.

Rather than pretending that fabricated readings are real, the system models the interface that real sensors would eventually provide.

This allows the complete software loop to be demonstrated honestly.

35. What the sensors actually do

The sensors are not used to classify every satellite pixel.

That would be scientifically unjustified.

A sensor node measures a point.

It cannot directly tell us the land condition 40 km away.

Instead, the prototype uses the nearest sensor as environmental context for a detected change.

The API asks:

During the same image-observation window, was the ground around the nearest sensor unusually dry?

This supports a simple seasonal assessment.

36. The seasonal sensor rule

The context endpoint uses the second satellite epoch:

2026-01-01 → 2026-04-30

as its sensor window.

This is intentional.

The relevant question is:

What were the ground conditions when the satellite observation period being interpreted occurred?

It should not depend on the date someone opens the dashboard.

The API therefore reads the epoch dates from `stats.json`.

37. Sensor assessment logic

A vegetation-loss event is considered:

likely_seasonal

when:

vegetation loss
AND
rainfall < 40 mm
AND
mean soil moisture < 18%

A conversion to Built-up is considered:

likely_permanent

because built surfaces do not normally disappear simply because the season changes.

Everything else is:

inconclusive

and should be sent for field verification.

These thresholds are deliberately simple.

They are not presented as a universal scientific model.

The production research version should calibrate them against real sensor data and independently verified changes.

38. Why nearest-sensor distance is calculated in Python

There are only a few sensor nodes.

Therefore the prototype calculates:

```
distance(change centroid, sensor)
```

using the Haversine formula.

The Haversine formula approximates great-circle distance on Earth.

This is enough for:

```
4 sensor nodes  
+  
hundreds of polygons
```

It avoids installing PostGIS simply to perform a small number of distance calculations.

For thousands or millions of geometries, this architecture should change.

39. Alert architecture

Every sufficiently large change event can become an alert.

The loader uses the transition and area to assign priority.

Examples:

```
Forest/Trees → Built-up  
Cropland → Built-up
```

receive high attention because they represent potentially significant land conversion.

The alert is not an accusation.

It means:

█ This location is worth verifying.

This is the correct role for an automated change-detection system.

40. Why prioritise alerts instead of showing everything equally?

A district can contain many detected changes.

A planner cannot manually inspect thousands of locations.

The useful output is therefore:

```
graph TD; A[all candidate changes] --> B[rank]; B --> C[highest-priority locations]; C --> D[field verification];
```

The system is therefore designed as a **screening and prioritisation system**, not an autonomous enforcement system.

41. Verification loop

The verification model is:

```
Satellite detects change
↓
System creates alert
↓
Human examines location
↓
Confirm / Reject / Unclear
↓
Database records verdict
↓
Precision score is calculated
```

The API accepts:

```
confirmed
rejected
unclear
```

and changes alert status accordingly.

This creates a feedback loop between automated detection and human verification.

42. Why verification matters

An automated model can be wrong.

The system should therefore measure:

```
precision =
confirmed /
(confirmed + rejected)
```

over verified alerts.

This is much more meaningful for an alerting system than simply reporting classifier accuracy.

A planner ultimately cares about:

If the system asks me to inspect something, how often is that alert actually useful?

43. Validation points

The pipeline exports validation points.

The intended workflow is:

1. Open the points in QGIS or Google Earth Pro.
2. Inspect the underlying high-resolution imagery.
3. Assign a human-interpreted reference class.
4. Compare the human class against the model.
5. Build an independent confusion matrix.

This gives a much stronger accuracy estimate than comparing the model only against Dynamic World.

44. PNG overlay architecture

The classified maps are exported as:

lulc_t1.png

lulc_t2.png

They are generated using Earth Engine thumbnail rendering.

The PNGs are not the primary scientific raster products.

They are a lightweight **visualisation representation**.

This was chosen because exporting full raster products from Earth Engine can hit memory/export limits in the prototype environment.

The map uses:

```
overlay_bounds.json
```

to position the PNG over the basemap.

45. Why EPSG:3857 is used for the visual overlays

The PNG thumbnail is rendered using:

```
EPSG:3857
```

which matches the common Web Mercator coordinate system used by web maps.

The overlay metadata stores the geographic corners separately.

This makes it straightforward for MapLibre to place the image over the basemap.

For scientific raster analysis, a projected CRS appropriate to the region should be used for area computations.

The web map CRS and scientific analysis CRS are separate concerns.

46. Why the frontend is a single HTML file

The frontend is intentionally:

```
api/static/index.html
```

rather than a React/Vite application.

Why?

The prototype does not need:

- Node.js
- npm
- bundling
- a frontend build pipeline
- dependency management
- a separate frontend server

FastAPI serves the HTML directly.

MapLibre provides the map rendering.

This reduces the number of moving parts.

The trade-off is maintainability.

For a production application, React/TypeScript would be a reasonable choice.

47. API architecture

FastAPI is the bridge between the database and browser.

The API has four broad responsibilities:

1. Read model results

Examples:

```
/api/stats  
/api/summary  
/api/changes  
/api/alerts
```

2. Provide geospatial data

/api/overlays
/api/changes
/api/alerts
/api/sensors

3. Provide AIoT context

/api/changes/{id}/context

4. Record human feedback

POST /api/changes/{id}/verify

This separation keeps the frontend from directly accessing SQLite.

48. API endpoints

Method	Endpoint	Purpose
--------	----------	---------

GET	/api/health	Check database and basic counts
-----	-------------	---------------------------------

GET	/api/overlays	Get LULC overlay bounds and legend
-----	---------------	------------------------------------

GET	/api/stats	Get class areas, model agreement and filter statistics
-----	------------	--

GET /api/summary Get grouped change transitions

GET /api/changes Get change polygons as GeoJSON

GET /api/changes/{id}/context Get nearest sensor and seasonal/permanent assessment

POST /api/changes/{id}/verify Store human verification

GET /api/verification/scorecard Calculate verified-alert precision

GET /api/alerts Get ranked alerts

GET /api/sensors Get sensor nodes and latest readings

GET /api/sensors/{id}/readings Get sensor time series

GET /api/report Generate the district report

GET /docs FastAPI interactive API documentation

49. File-by-file architecture

The repository is organised around a simple separation:

```
tn-lco/
|
├─ api/
|   ├── db.py
|   ├── main.py
|   ├── requirements.txt
|   └─ static/
|       └─ index.html
|
├─ db/
|   └─ schema_sqlite.sql
|
├─ gee/
|   └─ tn_lco_pipeline.py
|
├─ scripts/
|   ├── init_db.py
|   ├── load_results.py
|   └─ simulate_sensors.py
|
├─ data/
|   ├── lulc_t1.png
|   ├── lulc_t2.png
|   ├── overlay_bounds.json
|   ├── changes.geojson
|   ├── stats.json
|   ├── validation_points.geojson
|   └─ tnlco.db
|
└─ README.md
```

The development-only planning documents are intentionally not part of this README.

50. gee/tn_lco_pipeline.py

This is the **remote-sensing and machine-learning engine**.

It is designed to run in Google Colab with Earth Engine.

It contains the complete heavy processing chain:

```
configuration
↓
Sentinel-2 composite
↓
indices
↓
Sentinel-1 composite
↓
terrain
↓
feature stack
↓
Dynamic World labels
↓
Random Forest
↓
classification
↓
change detection
↓
filters
↓
vectorisation
↓
statistics
↓
validation points
↓
exports
```

Important functions/sections

mask_s2()

Removes unwanted optical observations.

s2_composite()

Creates the seasonal Sentinel-2 median composite and clear-observation count.

add_indices()

Adds NDVI, MNDWI, NDBI, NDMI and BSI.

s1_composite()

Creates the Sentinel-1 radar composite.

build_stack()

Combines optical, indices, radar and terrain.

dw_label()

Creates weak labels from Dynamic World.

train_and_classify()

Samples training data, trains Random Forest and returns the classified raster and confusion matrix.

Change detection block

Creates the raw change mask and applies the four filters.

Vectorisation block

Converts final change pixels into polygons and calculates area.

pixel_count()

Counts changed pixels for headline statistics.

class_areas()

Creates class-area histograms.

save_png()

Creates lightweight visual map images.

save_json()

Writes JSON/GeoJSON outputs.

51. GEE configuration parameters

These are the main parameters intentionally exposed near the top of the pipeline.

Parameter Current value Why

AOI_BBBOX Perundurai--Erode Defines the study window

AOI_NAME Perundurai-Erode window Human-readable study-area name

T1_START 2019-01-01 Baseline epoch

T1_END 2019-04-30 End of baseline seasonal window

T2_START 2026-01-01 Current epoch

T2_END 2026-04-30 End of current seasonal window

T2_CHECK_START 2025-07-01 Independent persistence-check season

T2_CHECK_END 2025-10-31 End of persistence-check season

SAMPLES_PER_CLASS 400 Balanced training sample size

RF_TREES 100 Random Forest ensemble size

MIN_MAPPING_UNIT_PIXELS 10 Removes very small spatial changes

THUMB_PIXELS 3072 Visual output size

52. Why RF_TREES = 100

Random Forest is an ensemble.

More trees generally stabilise the prediction, but also increase computation.

10 trees → fast but less stable

50 trees → reasonable

100 trees → current choice

500 trees → potentially stronger but unnecessary for prototype

100 is a practical middle ground.

It should be tuned experimentally if classifier performance becomes a research question.

53. Why THUMB_PIXELS = 3072

This parameter controls the size of the PNG visualisation.

It is not the spatial resolution of the underlying satellite analysis.

It controls the rendered image dimensions.

Increasing it gives:

- more visual detail
- larger files
- more server-side rendering work

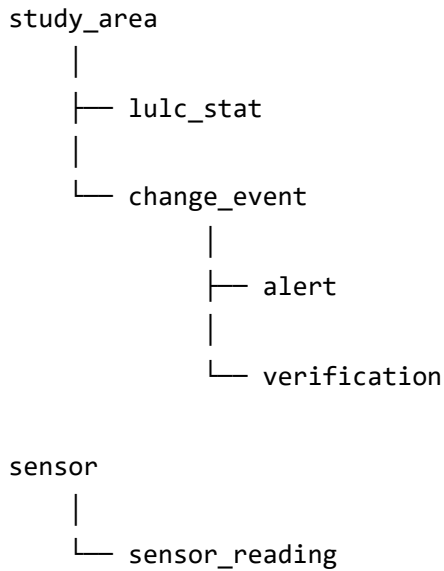
- higher risk of memory/export problems

The current value is a compromise after encountering Earth Engine thumbnail/export constraints.

54. db/schema_sqlite.sql

This defines the local database structure.

Conceptually the database contains:



The schema separates:

Static project information

study_area
lulc_class

Model output

lulc_stat
change_event

Operational prioritisation

alert

Human feedback

verification

AIoT context

sensor

sensor_reading

This separation is useful because a change event is not the same thing as an alert, and an alert is not the same thing as a human verification.

55. scripts/init_db.py

This script creates:

data/tnlco.db

from the SQLite schema.

It should normally be run before loading results.

Command:

```
python scripts/init_db.py
```

Why a separate initialisation script?

Database creation should not be hidden inside the API.

The API should consume a database.

The initialisation script owns database creation.

This keeps responsibilities separate:

```
init_db.py  
    → database structure
```

```
load_results.py  
    → model data
```

```
simulate_sensors.py  
    → sensor data
```

```
main.py  
    → API access
```

56. scripts/load_results.py

This is the bridge:

```
Earth Engine  
    ↓  
JSON / GeoJSON  
    ↓  
SQLite
```

It reads:

```
stats.json  
changes.geojson
```

and populates:

```
study_area  
lulc_stat  
change_event  
alert
```

It also computes:

```
centroid_lon  
centroid_lat
```

for each change geometry.

Why compute centroids during loading?

Because SQLite does not provide PostGIS-style geometry functions.

Precomputing the centroid allows the API to calculate nearest-sensor distance later without repeatedly parsing full polygons.

57. Idempotency of `load_results.py`

The loader clears model-output tables before reloading.

That means running:

```
python scripts/load_results.py
```

again does not simply append duplicate change events.

This is useful during development because the GEE outputs can be regenerated and reloaded repeatedly.

The trade-off is that this is not a production migration system.

A production database would require:

- versioned datasets

- dataset IDs
- timestamps
- model versions
- immutable processing runs

58. Geometry validation in the loader

The loader checks whether each feature has:

```
geometry
```

before attempting centroid calculation.

It also validates the transition code.

Invalid or unsupported features are skipped rather than crashing the entire import.

This matters because GeoJSON exports can contain:

```
"geometry": null
```

or unsupported geometry structures.

The prototype previously encountered exactly this export issue, so the loader is intentionally defensive.

59. scripts/simulate_sensors.py

This script provides the sensor side of the prototype.

It has two roles:

Seed history

```
python scripts/simulate_sensors.py --seed --days 730
```

This generates a long enough history for the 2026 image window.

Stream readings

```
python scripts/simulate_sensors.py --stream
```

This simulates a live stream into the database.

The data is always labelled as simulated.

60. Why seed 730 days?

The API's seasonal context is based on the actual second satellite epoch.

The sensor history therefore has to cover:

2026 Jan-Apr

A 730-day seed provides roughly two years of historical coverage.

The exact number is not scientifically required.

It is an engineering convenience that guarantees the required window exists.

61. api/db.py

This is the shared database utility module.

It owns:

- database path resolution
- SQLite connections
- row conversion
- Haversine distance
- centroid calculation

Important design principle:

`main.py` should not contain low-level database setup repeatedly.

Instead:

```
main.py
  ↓
api.db
  ↓
SQLite
```

This keeps API logic cleaner.

62. Why WAL mode is enabled

The connection uses:

```
PRAGMA journal_mode=WAL
```

WAL means:

Write-Ahead Logging

It improves the ability to read while another process writes.

That matters because the simulated sensor stream may write readings while FastAPI reads them.

Without WAL, SQLite can experience more reader/writer contention.

This is still a small local database, so WAL is enough for the prototype.

It is not a replacement for a production database.

63. Why foreign keys are enabled

The connection also uses:

```
PRAGMA foreign_keys=ON
```

This ensures SQLite actually enforces declared foreign-key relationships.

Without explicitly enabling foreign keys, SQLite can accept orphaned relationships depending on the schema.

This protects the integrity of:

```
change_event → alert  
change_event → verification  
sensor → sensor_reading
```

64. api/main.py

This is the application server.

It creates the FastAPI application and exposes the database through HTTP.

It also serves the frontend.

The module contains:

```
health()  
overlays()  
stats()  
summary()  
changes()  
change_context()  
alerts()  
verify()  
scorecard()  
sensors()  
readings()  
report()
```

The important architectural rule is:

```
| main.py does not run the Earth Engine model.
```

It only serves already-generated results.

That means the expensive geospatial computation and the web application are decoupled.

65. Why Earth Engine and FastAPI are separate

A common mistake would be to make the web server execute the satellite-processing pipeline whenever a user opens the website.

That would create:

```
browser
  ↓
FastAPI
  ↓
Earth Engine
  ↓
classification
  ↓
vectorisation
```

for every request.

That is slow, expensive, difficult to debug, and unnecessary.

TN-LCO instead uses:

```
offline/batch processing
  ↓
saved products
  ↓
API
  ↓
browser
```

This is a much better architecture for the current use case.

66. /api/health

Returns basic system status:

```
database exists
change-event count
sensor-reading count
```

This is intentionally simple.

A health endpoint should answer:

Is the application alive and does it have the expected core dependency?

It should not run a full scientific validation.

67. /api/overlays

Reads:

`data/overlay_bounds.json`

and returns the information required to place the LULC PNGs on the map.

The file contains:

- bounding box
- corner coordinates
- CRS metadata
- epoch labels
- class legend

This avoids hard-coding map coordinates into the frontend.

68. /api/stats

Combines database statistics with:

`stats.json`

It returns:

- class areas
- class names
- colours

- Dynamic World agreement
- raw change pixels
- retained change pixels
- scene counts

The separation is intentional:

SQLite

→ application-ready class statistics

stats.json

→ Earth Engine metadata and model metrics

69. /api/summary

Reads the database transition summary view.

It groups changes by:

```
from class
to class
number of polygons
total area
```

This is what powers the report's transition table.

70. /api/changes

Returns change polygons as:

GeoJSON FeatureCollection

Supported filters include:

```
min_area_ha  
to_class  
limit
```

Example:

```
/api/changes?min_area_ha=0.5
```

The API does not return unlimited data by default.

A limit protects the browser from accidentally loading an enormous GeoJSON document.

71. `/api/changes/{id}/context`

This is the AloT integration endpoint.

It:

1. finds the selected change event
2. loads sensor nodes
3. calculates distance from the change centroid to each node
4. selects the nearest sensor
5. reads rainfall and soil moisture during the relevant image window
6. applies the seasonal/permanent rule
7. returns the explanation

The endpoint therefore demonstrates an actual interaction between:

```
satellite-derived change  
+  
ground-condition context
```

rather than merely displaying sensor data on a separate chart.

72. /api/alerts

Returns ranked alert records.

Ordering is:

- priority
- then
- area

Lower priority number means higher priority.

The API also supports a status filter.

This allows the UI to distinguish:

- open
- confirmed
- rejected

alerts.

73. /api/changes/{id}/verify

This endpoint records human judgement.

Accepted verdicts:

- confirmed
- rejected
- unclear

The API stores:

change ID
verdict
note
observer
timestamp

The alert status is updated accordingly.

This turns the system from a one-way prediction pipeline into a verification loop.

74. /api/verification/scorecard

The scorecard calculates verified-alert precision.

Only the latest verification for each change event is considered.

This prevents repeated observations of the same event from artificially inflating the count.

The returned precision is:

$$\text{confirmed} / (\text{confirmed} + \text{rejected})$$

unclear is not counted as either success or failure.

75. /api/sensors

Returns the sensor nodes and their latest available readings.

The geometry is converted into GeoJSON points.

This makes sensor data directly usable by MapLibre.

76. /api/sensors/{id}/readings

Returns time-series readings for a selected sensor.

The API defaults to:

60 days

and allows up to:

365 days

This prevents an accidental request for an unlimited time series.

77. /api/report

The report is generated from the database.

It contains:

- study area
- compared epochs
- Sentinel-2 scene counts
- detected transitions
- total areas
- filter effect
- methodology limitations

The report intentionally includes limitations.

This is important because the system is a research prototype and the output should not appear more certain than it is.

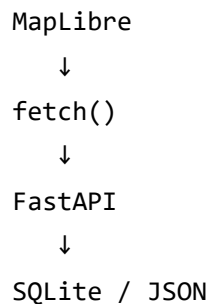
78. `api/static/index.html`

This is the web interface.

It is responsible for presentation and user interaction.

The frontend consumes API results rather than querying SQLite directly.

Conceptually:



This means the frontend does not need to know how the data is stored.

79. `api/requirements.txt`

This contains the Python dependencies required by the local API application.

The prototype intentionally keeps this list small.

The GEE notebook has a separate runtime because Earth Engine/Colab dependencies are not necessary for running the local web server.

This separation prevents the local installation from becoming unnecessarily large.

80. data/ directory

data/ contains generated runtime artifacts.

It is not source code.

Typical contents:

```
data/  
├─ lulc_t1.png  
├─ lulc_t2.png  
├─ overlay_bounds.json  
├─ changes.geojson  
├─ stats.json  
├─ validation_points.geojson  
└─ tnlco.db
```

These are generated by the pipeline and loader.

The directory should generally be excluded from version control because it contains generated datasets and the local database.

81. lulc_t1.png

The first epoch's classified LULC visualisation.

Current meaning:

2019 Jan-Apr

It is used for:

- visual map overlay
- human inspection
- demonstration

It is not intended to replace the full-resolution scientific raster product.

82. lulc_t2.png

The second epoch's classified LULC visualisation.

Current meaning:

2026 Jan-Apr

The frontend compares this with the first epoch.

83. overlay_bounds.json

Contains the map positioning metadata.

It prevents the frontend from having to know:

AOI coordinates
CRS
epoch labels
class colours

The backend can therefore provide the map configuration as data.

84. changes.geojson

This is the most important spatial output.

Each feature represents a retained change polygon.

Example structure:

```
{
  "type": "Feature",
  "geometry": {
    "type": "Polygon",
    "coordinates": [...]
  },
  "properties": {
    "transition": 23,
    "ndbi_d": 0.11,
    "vv_d": 1.2,
    "area_ha": 2.43
  }
}
```

The critical field is:

```
transition
```

which identifies the class conversion.

85. Why `.geo` must be exported

Earth Engine table exports can omit geometry if geometry is not selected.

The GeoJSON export therefore explicitly includes:

```
.geo
```

in its selectors.

Without it, the output can look like valid GeoJSON but contain:

```
"geometry": null
```

The loader cannot then calculate centroids or display spatial polygons.

This is an important implementation detail.

86. stats.json

This contains the model-level metadata.

It includes:

- AOI
- epoch dates
- scene counts
- class areas
- Dynamic World agreement
- confusion matrix
- raw change pixels
- retained change pixels
- class definitions

It is the bridge between the Earth Engine experiment and the local application.

87. validation_points.geojson

Contains sampled locations for independent visual validation.

The point itself is not ground truth.

The human must inspect the corresponding imagery and assign the reference class.

The file is therefore a **validation task dataset** rather than an automatically verified answer.

88. tnlco.db

This is the local SQLite database.

It contains the application-ready representation of:

- study area
- LULC statistics
- change events
- alerts
- sensors
- sensor readings
- verification results

It is generated locally.

Do not manually edit it unless debugging the prototype.

If it becomes corrupted, recreate it using:

```
python scripts/init_db.py
```

and reload the generated results.

89. Runtime sequence

The correct dependency order is:

1. Create Python environment
↓
2. Install local dependencies
↓
3. Initialise SQLite
↓
4. Seed simulated sensors
↓
5. Run Earth Engine pipeline
↓
6. Copy six generated outputs into data/
↓
7. Load results into SQLite
↓
8. Start FastAPI
↓
9. Open web interface

Commands:

```
python -m venv .venv
```

```
# Windows
```

```
.venv\Scripts\activate
```

```
pip install -r api/requirements.txt
```

```
python scripts/init_db.py
```

```
python scripts/simulate_sensors.py --seed --days 730
```

Run the GEE pipeline separately in Google Colab.

Then copy:

```
lulc_t1.png
```

```
lulc_t2.png
```

```
overlay_bounds.json
```

```
changes.geojson
```

```
stats.json
```

```
validation_points.geojson
```

into:

```
data/
```

Then:

```
python scripts/load_results.py
```

Finally:

```
uvicorn api.main:app --reload --port 8000
```

Open:

```
http://localhost:8000
```

API documentation:

```
http://localhost:8000/docs
```

90. What happens when the application starts

FastAPI:

```
creates app
  ↓
locates data/
  ↓
mounts /data
  ↓
mounts static frontend
  ↓
waits for HTTP requests
```

It does not:

- retrain the classifier
- download satellite imagery
- rebuild polygons
- regenerate sensors
- recompute the entire study

This makes startup fast.

91. What happens when a user opens the map

The browser loads:

```
api/static/index.html
```

The frontend then requests API data.

Typical flow:

GET /api/overlays



load map metadata

GET /api/stats



load statistics

GET /api/changes



load polygons

GET /api/alerts



load ranked changes

GET /api/sensors



load sensor nodes

When a user selects a change:

GET /api/changes/{id}/context

is used to retrieve environmental context.

92. Why the database is not queried by the browser

Allowing the browser to open SQLite directly would tightly couple the frontend to storage.

Instead:

Frontend
↓ HTTP
FastAPI
↓ SQL
SQLite

This gives a clean separation.

A future migration from:

SQLite

to:

PostGIS

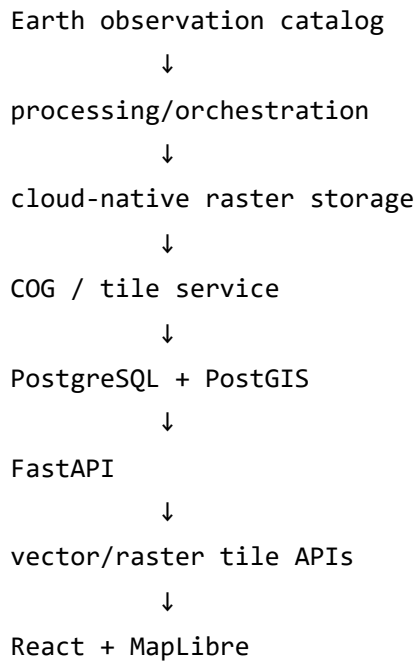
can then happen largely behind the API.

93. Prototype vs production architecture

Current prototype

Earth Engine
↓
JSON/GeoJSON
↓
SQLite
↓
FastAPI
↓
MapLibre

More realistic production architecture



The production architecture should also add:

- authentication
- authorisation
- audit logs
- dataset versioning
- spatial indexes
- automated processing jobs
- monitoring
- backups
- real sensor ingestion
- proper field-validation data
- administrative boundary layers
- uncertainty estimates
- model versioning

Those are not required to prove the prototype's core idea.

94. Why PostGIS is the obvious future database

The current SQLite approach deliberately avoids infrastructure.

But a state-scale GIS system needs spatial database capabilities.

PostGIS would allow:

```
ST_Intersects(...)
ST_DWithin(...)
ST_Area(...)
ST_Transform(...)
ST_MakeValid(...)
```

and spatial indexes.

That enables queries such as:

```
Which change polygons intersect this village?
```

```
Which changes occurred within 500 m of a water body?
```

```
Which blocks have the greatest built-up expansion?
```

```
Which alerts are inside a protected area?
```

SQLite is not the wrong database for the prototype.

It is the wrong database for a large spatial production system.

That distinction is important.

95. Why the prototype does not use a Digital Twin

The original proposal describes a Digital Twin.

A true Digital Twin normally implies a strong relationship between a physical system and a continuously updated digital representation.

This prototype does not have:

- live statewide sensor coverage
- live land-state synchronisation
- validated dynamic simulation
- two-way operational integration

Therefore the prototype does not claim to implement a Digital Twin.

A future version could implement a **scenario simulator**:

```
current LULC
  +
transition model
  +
scenario parameters
  ↓
simulated future map
```

That is a much more defensible first step.

96. Why the mobile application is not implemented

The original proposal includes a mobile application.

For this prototype, the verification workflow can be demonstrated through the WebGIS.

A future mobile field-verification client could:

```
receive alert
  ↓
navigate to location
  ↓
capture observation
  ↓
attach photo
  ↓
confirm/reject
  ↓
sync to API
```

This is more useful than a generic public information app because it directly closes the model-validation loop.

97. Why state-wide coverage is not implemented

The current AOI is approximately:

38 × 33 km

rather than all of Tamil Nadu.

Scaling from one window to the whole state changes:

- processing volume
- storage requirements
- tile generation
- database size
- validation requirements
- administrative boundary management
- model transferability
- monitoring infrastructure

A district-scale prototype is therefore a sensible architectural unit.

The pipeline can later be parameterised by district.

98. Known limitations

98.1 Dynamic World is not ground truth

Agreement with Dynamic World is not independent accuracy.

98.2 Spatial validation is still required

Random pixel train/test splits can overestimate performance because nearby pixels are correlated.

98.3 Minimum mapping unit

Approximately 0.1 ha is the current lower practical mapping threshold.

Individual small buildings are not reliably resolved by 10 m imagery.

98.4 One study area

The current experiment is spatially narrow.

Performance in another district is not automatically guaranteed.

98.5 Simulated sensors

All sensor readings are simulated.

No hardware is deployed.

98.6 No cadastral data

The system cannot make parcel-level or legality decisions.

98.7 No authentication

The prototype is intended for local use.

It should not be exposed publicly without authentication and security controls.

98.8 PNG overlays

The web map uses PNG overlays for lightweight visualisation.

At high zoom levels, resampling artefacts can become visible.

A production version should serve properly tiled raster data.

98.9 SQLite spatial limitations

There is no native spatial index or PostGIS spatial query engine.

Nearest-sensor calculations are therefore done in Python.

99. What is real and what is simulated?

Component Status

Sentinel-2 imagery Real

Sentinel-1 imagery Real

SRTM elevation/slope Real

Random Forest Real model execution

Dynamic World labels Real external product, but
model-generated

Change polygons Real computed outputs

LULC areas Real computed outputs

Confusion matrix Real computed agreement metric

Validation points Real generated sample locations;
human interpretation still required

Sensor readings **SIMULATED**

Alert software Real

Human verification loop Real software workflow

Cadastral records Absent

Ownership data Absent

Zoning data Absent

Digital Twin Not implemented

Mobile application Not implemented

State-wide coverage Not implemented

Real-time satellite ingestion Not implemented

This distinction should remain visible in demonstrations.

100. Why the system is GeoAI

GeoAI is not simply:

AI + map

The defining relationship here is:

```
geospatial observations
    ↓
machine learning
    ↓
spatially referenced predictions
```

The classifier receives geospatially located satellite-derived features and produces a spatially located LULC map.

The change detector then performs spatial analysis on those predictions.

The final result is not merely:

class = Built-up

It is:

```
this geographic region
was classified as
Cropland → Built-up
```

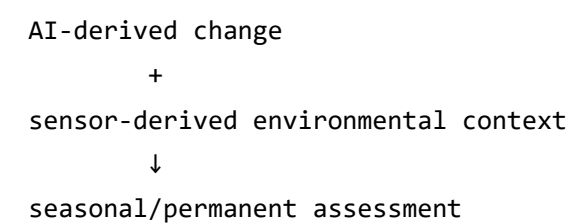
That spatial relationship is fundamental to the problem.

101. Why the system is only a prototype AloT system

The sensor component provides the "IoT" side.

The AI component produces land-cover and change predictions.

The integration is:



That is a meaningful integration.

However, because the sensors are simulated, this should be described as:

| an AloT-oriented prototype or AloT workflow demonstration

rather than a deployed AloT monitoring network.

102. Architectural decisions at a glance

Decision Choice Main reason Alternative

EO platform Earth Engine Avoid local bulk STAC +
processing rasterio/xarray

Optical source Sentinel-2 Free + 10 m Landsat, commercial imagery

Radar Sentinel-1 Cloud-independent No radar supporting signal

Terrain SRTM Free terrain Other DEMs context

Composite Median Robust to outliers Mean, single date

LULC model Random Forest Fast + U-Net, DeepLab, ViT interpretable baseline

Labels Dynamic World Free weak labels Human-labelled dataset

Change detection Post-classification Explainable Deep change network

Built-up filter NDBI + VV Independent Class-only change corroboration

Seasonal filter Cross-season Suppress crop-cycle Single-date persistence false positives comparison

Mapping unit 0.1 ha Remove tiny noise Larger/smaller threshold

Database SQLite Zero infrastructure PostgreSQL/PostGIS

Geometry storage GeoJSON text SQLite-compatible PostGIS geometry

API FastAPI Small + typed + Flask, Django fast

Frontend Single HTML + No build step React + MapLibre MapLibre

Sensor distance Haversine in Python Few nodes PostGIS spatial query

Raster delivery PNG overlay Lightweight COG + tile server
prototype

Validation Human reference Independent check DGPS/UAV field
points survey

**Sensor data Simulated Hardware Real IoT deployment
unavailable**

103. The core engineering philosophy

The project follows five principles.

1. Prefer a complete simple pipeline over many disconnected technologies

A working:

satellite → model → change → API → map

is more valuable than ten technologies that never exchange real outputs.

2. Every major filter should have a reason

For example:

0.1 ha

exists to remove small spatial noise.

NDBI + VV

exists to corroborate built-up conversion.

cross-season persistence

exists to reduce seasonal false alarms.

5 clear observations

exists to avoid low-quality comparisons.

3. Do not hide uncertainty

Dynamic World agreement is not ground truth.

Simulated sensors are not real sensors.

A detected change is not an illegal conversion.

The software should make these distinctions visible.

4. Use the simplest infrastructure that proves the idea

SQLite is enough for the current prototype.

PostGIS is appropriate later.

5. Make the system replaceable

The interfaces are deliberately separated.

For example:

Earth Engine

could eventually be replaced by:

STAC + rasterio/xarray

without redesigning the API.

Likewise:

SQLite

could later become:

PostGIS

without requiring the frontend to know.

104. Suggested future evolution

The natural evolution is:

Phase 1 --- Current prototype

One study window

+

Sentinel-1/2

+

Random Forest

+

change filters

+

simulated sensors

+

SQLite

+

FastAPI

+

MapLibre

Phase 2 --- Scientific strengthening

Add:

- independent field labels
- +
- spatial block validation
- +
- uncertainty estimates
- +
- threshold calibration
- +
- more historical epochs

Phase 3 --- Better prediction

Add:

- transition probabilities
- +
- suitability model
- +
- cellular allocation
- +
- hindcast validation

Phase 4 --- Real IoT

Replace:

- simulated readings

with:

- real sensor nodes
- +
- MQTT
- +
- buffering
- +
- time-series storage

Phase 5 --- Production GIS

Replace:

SQLite

with:

PostgreSQL + PostGIS

and replace:

PNG

with:

Cloud-Optimized GeoTIFF

+

tile service

Phase 6 --- Multi-district operation

Run the same processing pipeline independently for:

district

↓

district products

↓

central catalogue

rather than immediately attempting a monolithic statewide processing job.

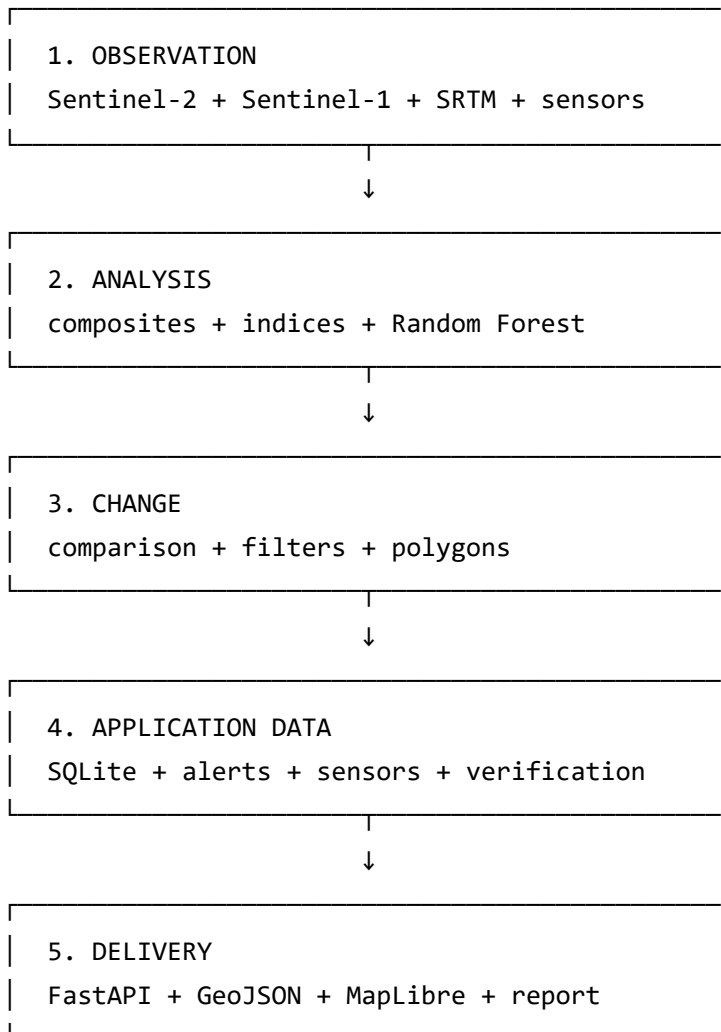
105. The one-sentence description

If someone asks what TN-LCO is:

TN-LCO is a district-scale GeoAI prototype that classifies land cover from free Sentinel imagery, detects persistent land-cover changes, uses environmental context to reduce seasonal false alarms, and turns the results into ranked field-verification alerts through a lightweight FastAPI and MapLibre application.

106. Final mental model

The entire repository can be understood as five layers:



If those five layers work together, the prototype has achieved its main purpose.

If a future version adds more technologies, each addition should answer one question:

What problem does this component solve that the current architecture cannot solve adequately?

If there is no clear answer, the component probably does not belong in the system yet.